

SQL Server Roadmap — 80/20, Hands-on, Scenario-Driven

Database Learning · Phase 1 (SQL Server / T-SQL) → Phase 2 (MongoDB, gated)

Who this is for: a full-stack Angular + .NET developer who already knows basic SELECT, INSERT and CREATE. The goal is SQL that is correct, fast and safe under concurrency, plus the judgment to handle the scenarios that actually come up in real apps and senior interviews.

How it's cut: each topic is tagged by how much of real-world usage it covers.

Tag	Meaning	Treatment
CORE Core	The 20% you'll use 80% of the time	Gen: , full lab, drilled, in the PDF
KNOW Working knowledge	Comes up regularly; use it correctly when needed	Gen lite: , one lab task
AWARE Awareness	Know the term and when someone else owns it	One awareness sheet at the end
GATE Gate	Must be solid before Phase 2 (MongoDB) starts	Joins, windows, indexing, transactions

Pace: at 10–15 hrs/week, Phase 1 takes roughly 8–10 weeks. A **CORE** topic is about 1.5–2 hrs (read, lab, review). A module is one chat.

What changed from the old SQL_Learning_Order.md:

- Indexing moved from last to M7, directly after the querying modules, because the gate depends on it.
- Programmability (procs, functions, triggers, views) was collapsed from six sections into one module.
- Added topics that were missing: logical processing order, outer-join filter placement, join fan-out, APPLY, sargability and implicit conversion, covering and filtered indexes, statistics, keyset pagination, RCSI/SNAPSHOT, optimistic concurrency, race-condition patterns, OUTPUT, the MERGE caveats, XACT_ABORT, parameter sniffing, and an EF Core bridge module.
- Every module now ends in a scenario pack and a production-incident capstone.
- WHILE loops, cursors, the GCP/BigQuery bridge and DBA topics were moved to awareness level.

The practice database: RetailOrderDb

One schema is used throughout. It's small enough to hold in your head and rich enough to produce every classic problem: NULLs, fan-out, hierarchies, time ranges, races and skew. You build it yourself in M1.

Customers	(CustomerId PK, Email UQ, FullName, Country, CreatedAt, IsActive)
Addresses	(AddressId PK, CustomerId FK, Line1, City, PostalCode, IsDefault)
Categories	(CategoryId PK, Name, ParentCategoryId FK→Categories NULL) -- tree
Products	(ProductId PK, CategoryId FK, Sku UQ, Name, Price DECIMAL(10,2), StockQty, IsDiscontinued, RowVer ROWVERSION)
Employees	(EmployeeId PK, FullName, ManagerId FK→Employees NULL, HireDate)
Orders	(OrderId PK, CustomerId FK, EmployeeId FK NULL, OrderDate DATETIME2, Status, ShippedDate DATETIME2 NULL, ShippingAddressId FK)
OrderItems	(OrderId FK, ProductId FK, Quantity, UnitPrice) -- PK(OrderId,ProductId)
Payments	(PaymentId PK, OrderId FK, Amount, PaidAt, Method) -- 0..n per order

Seed volume matters. With 20 rows, every query is instant and every plan is a scan, so you learn nothing about performance. The M1 lab writes a set-based seed script that produces about 10k customers, 1k products and 100k orders, with realistic skew (a few heavy customers, seasonal months, some NULL `ShippedDates`, a few duplicate-looking customers). All later labs and self-checks assume this data.

Feedback loop, used from M2 onward: `SET STATISTICS IO, TIME ON`; plus the actual execution plan (Ctrl+M in SSMS). Logical reads are your scoreboard.

M0 — Setup & Baseline (1 session)

- 0.1 **CORE** Environment: SQL Server Developer Edition (free), plus SSMS or VS Code with the mssql extension. Create `RetailOrderDb`.
- 0.2 **CORE** The measurement toolkit: `STATISTICS IO/TIME`, estimated vs actual plan, and how to clear the cache in dev only.
- 0.3 **CORE** Diagnostic (6 questions: NULL comparison, alias in WHERE, LEFT JOIN + WHERE, NOT IN + NULL, COUNT variants, BETWEEN on datetime2). A clean score lets you skim M2.

M1 — Schema Design & Data Types (builds the lab)

- 1.1 **CORE** Data types that bite:
 - DECIMAL for money, not FLOAT (MONEY also has pitfalls)
 - NVARCHAR vs VARCHAR
 - DATETIME2 vs DATETIME
 - DATETIMEOFFSET and UTC storage
 - BIT, and sizing columns
- 1.2 **CORE** Keys: surrogate vs natural; IDENTITY vs GUID (NEWID fragmentation vs NEWSEQUENTIALID); composite keys.
- 1.3 **CORE** Constraints as business rules: NOT NULL, DEFAULT, CHECK, UNIQUE, FK with ON DELETE choices; a filtered unique index for "only one default address".
- 1.4 **CORE** Relationships: one-to-many, many-to-many junction tables, self-references (hierarchies).
- 1.5 **CORE** Normalization to 3NF by spotting update/insert/delete anomalies. Deliberate denormalization, and why snapshotting `UnitPrice` into `OrderItems` is correct modelling rather than denormalization.
- 1.6 **KNOW** Scripts that can run twice: `IF NOT EXISTS`, `CREATE OR ALTER`. Hand-written scripts vs EF Core migrations.
- 1.7 **KNOW** Audit columns, soft delete (and the query tax it adds), temporal tables (awareness).

Lab: write `01_schema.sql` and `02_seed.sql` (set-based, no loops) for RetailOrderDb.

Scenario pack themes:

- store prices and currency correctly
- email must be unique but changeable
- keep the order total even if the product price changes
- a product is discontinued but its old orders must remain
- a customer has many addresses, exactly one default

M2 — How SQL Server Reads Your Query

- 2.1 **CORE** **Logical query processing order:** FROM → ON → JOIN → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → TOP/OFFSET. Explains alias errors, HAVING vs WHERE, and ORDER BY by alias.
- 2.2 **CORE** **NULL and three-valued logic:**
 - `= NULL` vs `IS NULL`
 - `<>` silently excluding NULLs
 - COALESCE vs ISNULL
 - NULLs in aggregates and in sorting
- 2.3 **CORE** Filtering correctly:
 - half-open date ranges (`>= start AND < next`)
 - LIKE and escaping wildcards
 - IN lists
 - CASE in SELECT vs WHERE
- 2.4 **CORE** Sorting and paging basics: non-deterministic ORDER BY (ties), TOP WITH TIES, OFFSET/FETCH.
- 2.5 **KNOW** The functions to know cold:
 - dates: DATEADD, DATEDIFF, EOMONTH, DATETRUNC (2022+)
 - strings: CONCAT, TRIM, STRING_SPLIT
 - conversion: TRY_CONVERT and CAST

Scenario pack themes:

- orders from last calendar month
- search products by partial name
- page 3 of the catalogue sorted by price
- customers with no phone

M3 — Joins **GATE**

- 3.1 **CORE** INNER, LEFT, FULL and CROSS, reasoned by what each does to the **row count**.
- 3.2 **CORE** **ON vs WHERE in outer joins:** how a WHERE filter silently turns a LEFT JOIN into an INNER JOIN.
- 3.3 **CORE** **Join fan-out:** joining two one-to-many children multiplies rows. Always know the grain of the result.
- 3.4 **CORE** Multi-table join paths and self-joins (employee → manager). Joining on non-key columns.
- 3.5 **KNOW** Physical joins at plan-reading level: nested loops, hash and merge, and what each tells you.

Scenario pack themes:

- all customers with their order count, including those with zero
- products never sold
- employee with manager name
- an order-detail report with totals that don't match (fan-out bug)

M4 — Aggregation & Reporting

- 4.1 **CORE** GROUP BY and HAVING; COUNT(*) vs COUNT(col) vs COUNT(DISTINCT); aggregates and NULLs; AVG on integers.
- 4.2 **CORE** **Aggregate at the right grain**: pre-aggregate in a derived table or CTE before joining, which fixes double counting.
- 4.3 **CORE** Conditional aggregation, `SUM(CASE WHEN ...)`, for crosstab reports and dashboard KPIs in one pass.
- 4.4 **KNOW** STRING_AGG, and ROLLUP/GROUPING SETS for subtotals.
- 4.5 **KNOW** Time-bucketed reports: a calendar/numbers table so months with zero sales still appear.

Scenario pack themes:

- revenue per month including zero months
- average order value by country
- orders paid in full vs partially paid (Payments is 0..n per order)
- top category per quarter

M5 — Subqueries, CTEs & Set Operations

- 5.1 **CORE** Scalar, derived-table and correlated subqueries, and when each reads best.
- 5.2 **CORE** **Semi-joins and anti-joins**: EXISTS / NOT EXISTS vs IN / JOIN. The NOT IN + NULL trap.
- 5.3 **CORE** CTEs for readability. A CTE is not materialized; referencing it twice runs it twice.
- 5.4 **KNOW** Recursive CTEs for the category breadcrumb and the org chart, including MAXRECURSION and cycle guarding.
- 5.5 **KNOW** UNION vs UNION ALL (the cost of dedup), EXCEPT and INTERSECT for reconciliation queries.

Scenario pack themes:

- customers who bought from category A but never from B
- full category path for each product
- reconciling two lists (e.g. imported file vs table)

M6 — Window Functions & APPLY **GATE**

- 6.1 **CORE** OVER and PARTITION BY vs GROUP BY: keeping detail rows while computing group values.
- 6.2 **CORE** ROW_NUMBER, RANK and DENSE_RANK:
 - **top-N per group**
 - **deduplication** (keep newest, delete the rest)

- how ties behave in each
- 6.3 **CORE** LAG and LEAD: period-over-period change and time between events.
- 6.4 **CORE** Running totals and moving averages. Frame clauses, and the **default RANGE frame** trap (use ROWS).
- 6.5 **CORE** **CROSS APPLY** / **OUTER APPLY**: "latest N orders per customer", and when APPLY beats ROW_NUMBER.
- 6.6 **KNOW** Gaps and islands: consecutive-day streaks and missing sequence numbers.
- 6.7 **KNOW** FIRST_VALUE / LAST_VALUE (frame trap), NTILE, PERCENT_RANK.

Scenario pack themes:

- latest order per customer
- top 3 products per category by revenue
- month-over-month growth %
- remove duplicate customers, keeping the oldest
- customers who ordered 3 months in a row

M7 — Indexing & Execution Plans **GATE**

- 7.1 **CORE** Mental model: 8KB pages, B-trees, heap vs clustered index, and what a "row lookup" physically costs.
- 7.2 **CORE** Choosing the clustered key: narrow, unique, static, ever-increasing. Why random GUIDs hurt.
- 7.3 **CORE** Nonclustered and composite indexes. **Column order** (equality columns, then range, then sort). Key lookups.
- 7.4 **CORE** **Covering indexes** (INCLUDE) and **filtered indexes**. When a covering index isn't worth its write cost.
- 7.5 **CORE** **Sargability**:
 - functions on columns
 - leading wildcards
 - OR across columns
 - **implicit conversion**, e.g. an NVARCHAR parameter against a VARCHAR column (a classic EF Core cause)
- 7.6 **CORE** Reading plans: seek vs scan vs lookup, estimated vs actual rows, the fattest arrow, STATISTICS IO. **Statistics**: what they are and what stale stats do.
- 7.7 **KNOW** Index costs: write amplification, over-indexing, why "missing index" hints can't be trusted blindly, indexing foreign keys.
- 7.8 **KNOW** **Keyset (seek) pagination** vs OFFSET at deep pages.

Scenario pack themes:

- orders-by-customer page slow since data grew
- product search slow
- "the index exists but isn't used"
- a report that's fine in dev and slow in prod
- choose indexes for a new endpoint, given its queries

M8 — Transactions & Concurrency GATE

- 8.1 **CORE** ACID in practice; transaction scope; autocommit; `SET XACT_ABORT ON`; keeping transactions short.
- 8.2 **CORE** Locks: S, X and U; blocking vs deadlock; lock escalation (awareness).
- 8.3 **CORE** Isolation levels mapped to the anomalies each allows: dirty read, non-repeatable read, phantom, **lost update**.
- 8.4 **CORE** **RCSI and SNAPSHOT** (row versioning), and why `NOLOCK` is not a performance fix.
- 8.5 **CORE** **Optimistic concurrency** (ROWVERSION check) vs **pessimistic** (`UPDLOCK`, `HOLDLOCK`). Choosing between them.
- 8.6 **CORE** Race-condition patterns:
 - the last item in stock (atomic `UPDATE ... WHERE StockQty >= @q`)
 - check-then-insert duplicates
 - idempotent order submission
- 8.7 **KNOW** Deadlocks: reading the deadlock graph, consistent access order, retry logic.

Scenario pack themes:

- two users buy the last unit
- a double-clicked "Place order"
- two admins edit the same product
- a report blocks checkout

M9 — Data Modification Patterns

- 9.1 **CORE** `INSERT...SELECT`; `UPDATE` and `DELETE` with `JOIN`. The **safe prod-change routine**: `SELECT` first, `BEGIN TRAN`, check `@@ROWCOUNT`, then `COMMIT` or `ROLLBACK`.
- 9.2 **CORE** The **OUTPUT** clause (capturing inserted IDs and before/after values); `SCOPE_IDENTITY` vs `@@IDENTITY`.
- 9.3 **CORE** Upserts: `UPDATE`-then-`INSERT` with the right locking vs **MERGE**, including its known caveats and when it's fine.
- 9.4 **KNOW** Batching large updates and deletes: chunked loops, log growth, lock escalation.
- 9.5 **KNOW** `TRUNCATE` vs `DELETE`; staging-table sync from an imported file.

Scenario pack themes:

- apply a price change from a supplier feed
- archive orders older than 3 years without blocking the site
- fix wrongly shipped statuses safely in prod
- return the new OrderId and line IDs to the API

M10 — Stored Procedures, Functions, Views & Triggers

- 10.1 **CORE** Stored procedures: parameters, `OUTPUT` params, result sets, `SET NOCOUNT ON`, and when a proc is justified vs app code.
- 10.2 **CORE** **Error handling template**: `TRY/CATCH` + `THROW` + `XACT_ABORT` + `@@TRANCOUNT` check. Why `RAISERROR` is legacy.
- 10.3 **CORE** **Parameter sniffing**:
 - the "fast in SSMS, slow from app" symptom
 - fixes: `OPTION(RECOMPILE)`, `OPTIMIZE FOR`, and query redesign

- 10.4 **KNOW** Functions: inline TVFs are good; scalar and multi-statement functions are slow (with scalar UDF inlining in 2019+ as the exception).
- 10.5 **KNOW** Views as an API or security layer: no ORDER BY, no parameters. Indexed views (awareness).
- 10.6 **KNOW** Triggers for audit trails. The multi-row trap (a trigger fires once per statement). When not to use them.
- 10.7 **KNOW** Temp tables vs table variables. Dynamic SQL with `sp_executesql` for optional-filter search, and SQL injection.
- 10.8 **KNOW** JSON in SQL Server: FOR JSON and OPENJSON for API-shaped results and bulk payloads.

Scenario pack themes:

- a `PlaceOrder` proc (validate, reserve stock, insert, all-or-nothing)
- audit every price change
- a search endpoint with 6 optional filters
- a proc that went slow overnight

M11 — EF Core ↔ SQL Server Bridge

- 11.1 **CORE** Seeing the SQL: `ToQueryString()`, logging, and matching it to a plan.
- 11.2 **CORE** N+1 queries; Include vs projection; **cartesian explosion** and `AsSplitQuery`.
- 11.3 **CORE** Tracking vs `AsNoTracking`; projecting to DTOs; `IQueryable` vs `IEnumerable` (client-side filtering).
- 11.4 **CORE** Raw SQL safely: `FromSql` (interpolated, parameterized) vs `FromSqlRaw`; calling procs.
- 11.5 **CORE** Concurrency tokens (`[Timestamp]` / rowversion) and `DbUpdateConcurrencyException` end to end.
- 11.6 **KNOW** Transactions in EF Core; `SaveChanges` batching; `ExecuteUpdate` / `ExecuteDelete`.
- 11.7 **KNOW** Indexes and string mapping via Fluent API (`IsUnicode(false)` to avoid implicit conversion); migrations hygiene.

Scenario pack themes:

- an API endpoint issuing 201 queries
- a list page loading 40MB
- an edit form silently overwriting another user's change
- a LINQ filter on a VARCHAR column that scans

M12 — Capstone: Production Scenarios & Phase 1 Exit

- **Incident drills** (you debug, Claude plays the system):
 - a dashboard double-counts revenue
 - checkout deadlocks under load
 - search times out
 - duplicate customers appear
 - the report is off by one day (timezone)
 - page 5,000 of the API is slow
 - a proc is fast in SSMS but slow from the app
 - a bulk price update blocks the site

- **Design exercise:** add Coupons/Promotions and Product Reviews. Cover the schema, constraints, indexes for the stated queries, and concurrency concerns.
- **Mock interview:** a 45-minute mixed round.
- **Gate check** → Phase 2.

AWARE Awareness sheet (one session, at the end)

- Backup/restore and recovery models
 - Table partitioning
 - Replication and Always On AGs
 - Query Store (worth opening once)
 - Columnstore indexes (analytics)
 - Full-text search
 - PIVOT/UNPIVOT
 - Cursors and WHILE loops (why set-based wins)
 - GRANT/DENY and least-privilege app logins
 - CLR and Service Broker
 - Linked servers
 - Cloud warehouses (BigQuery/Synapse): columnar storage, pay-per-scan, partition pruning
-

Phase 1 → Phase 2 gate

All four must be true before Phase 2 starts:

1. **M3 Joins:** you can predict the row count of any join on RetailOrderDb before running it, and you can spot fan-out.
2. **M6 Windows:** you solve top-N-per-group, dedup, running totals and period-over-period without reference.
3. **M7 Indexing:** given a query, you design the index, predict the plan shape, and confirm it with STATISTICS IO. You can explain a non-sargable predicate.
4. **M8 Transactions:** you can fix a lost-update and a last-item race, and you can choose optimistic vs pessimistic with a reason.

The check is a **Drill: mixed** score of at least 80%, plus 3 of the M12 incident drills solved without hints.

Phase 2 — MongoDB (outline; detailed roadmap written when the gate opens)

- P2-M1 Document modeling: embedding vs referencing, and designing from access patterns. RetailOrderDb gets remodelled as documents.
- P2-M2 CRUD and query operators; projections; arrays.
- P2-M3 Aggregation pipeline: `$match`, `$group`, `$lookup` (and its cost), `$unwind`, `$facet`, and window-style `$setWindowFields`.
- P2-M4 Indexes: compound indexes with the ESR rule, multikey, TTL, `explain()`.
- P2-M5 Schema validation and schema evolution.
- P2-M6 Consistency: write/read concerns, multi-document transactions and their real limits.

- P2-M7 The .NET driver (and the EF Core provider) from the app side.
 - P2-M8 **SQL vs Mongo decision drills**: the operations-to-tool rule applied to engine choice.
-

Repo layout

Database/	
SQL_Server_Learning_Path/	
README.md	how to use this path
Roadmap/	this file (.md + .pdf)
Prompts/	Database_Master_Prompts.md
Labs/setup/	01_schema.sql, 02_seed.sql (built in M1)
Labs/M03/	your ticket solutions per module
Modules/M03/	consolidated topic .md files
PDFs/	SQL_M03_Joins.pdf ...
SQL_Learning_Path_AI-main/	original roadmap (kept for history)